black_scholes_model_UPDATED_JOB_C

April 25, 2026

1 Black–Scholes Modeling and Dynamic Delta Hedging Layer

This notebook implements the modeling and hedging-simulation deliverables for the delta-hedging project. It loads the cleaned project data, computes Black–Scholes theoretical option values and deltas over each option episode, compares implied-volatility and realized-volatility inputs, then implements dynamic delta hedging with discrete rebalancing, cash-account accrual, transaction-cost controls, and portfolio P&L tracking.

[3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from scipy.stats import norm

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)
```

1.1 1. Portable project paths

[4]:
```
PROJECT_ROOT = Path.cwd().resolve()

while PROJECT_ROOT != PROJECT_ROOT.parent and not (PROJECT_ROOT / "data" /␣
↪"output").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_PATH = PROJECT_ROOT / "data" / "output"

if not DATA_PATH.exists():
    raise FileNotFoundError(
        f"Could not find data/output from current directory. Current resolved␣
↪root: {PROJECT_ROOT}"
    )

print("Project root:", PROJECT_ROOT)
print("Data path:", DATA_PATH)
print("Available files:")
for f in sorted(DATA_PATH.iterdir()):
    print(" -", f.name)
```

```
Project root: C:\Users\elain\Downloads\5010-Project
Data path: C:\Users\elain\Downloads\5010-Project\data\output
Available files:
 - bs_delta_panel.csv
 - bs_delta_panel.parquet
 - bs_delta_panel_README.md
 - data_quality_report.md
 - dividends.parquet
 - episodes.parquet
 - modeling_outputs_manifest.txt
 - option_chain_snapshot.parquet
 - report_figures
 - report_tables
 - risk_free.parquet
 - stock_intraday_15m.parquet
 - stock_intraday_1h.parquet
 - stock_prices.parquet
 - vix.parquet
```

1.2 2. Load cleaned data

Main inputs: - `episodes.parquet`: the 12 option experiments. - `stock_prices.parquet`: SPY daily prices and realized-volatility columns. - `risk_free.parquet`: daily risk-free rate in decimal form.

[5]:
```
episodes = pd.read_parquet(DATA_PATH / "episodes.parquet")
prices = pd.read_parquet(DATA_PATH / "stock_prices.parquet")
rf = pd.read_parquet(DATA_PATH / "risk_free.parquet")
```

```python
print("Episodes:", episodes.shape)
print("Prices:", prices.shape)
print("Risk-free:", rf.shape)

display(episodes.head())
display(prices.head())
display(rf.head())
```

```
Episodes: (12, 18)
Prices: (1583, 11)
Risk-free: (1583, 2)
```

```
         episode_id  regime ticker cp_flag moneyness inception_date          S0 ␣
↪strike expiration  T_days   T_years  \
0    SPY_calm_C_ATM    calm    SPY       C       ATM     2021-03-01  389.579987 ␣
↪ 390.0 2021-05-28      63  0.250000
1    SPY_calm_C_OTM    calm    SPY       C       OTM     2021-03-01  389.579987 ␣
↪ 409.0 2021-05-28      63  0.250000
2    SPY_calm_P_ATM    calm    SPY       P       ATM     2021-03-01  389.579987 ␣
↪ 390.0 2021-05-28      63  0.250000
3  SPY_stress_C_ATM  stress    SPY       C       ATM     2020-02-03  324.119995 ␣
↪ 324.0 2020-05-01      62  0.246032
4  SPY_stress_C_OTM  stress    SPY       C       OTM     2020-02-03  324.119995 ␣
↪ 340.0 2020-05-01      62  0.246032

   iv_inception_vix  iv_inception_vix3m  r_inception  div_amount_in_window␣
↪div_ex_dates  pv_div_inception  q_inception
0            0.2335              0.2678      0.00028                 1.278  ␣
↪2021-03-19          1.277982     0.013143
1            0.2335              0.2678      0.00028                 1.278  ␣
↪2021-03-19          1.277982     0.013143
2            0.2335              0.2678      0.00028                 1.278  ␣
↪2021-03-19          1.277982     0.013143
3            0.1797              0.1804      0.01518                 1.406  ␣
↪2020-03-20          1.403313     0.017636
4            0.1797              0.1804      0.01518                 1.406  ␣
↪2020-03-20          1.403313     0.017636

        date ticker        open        high         low       close    volume  ␣
↪adj_close    logret  rv_20d  rv_60d
0 2020-01-02    SPY  323.540009  324.890015  322.529999  324.869995  59151200 ␣
↪296.888123       NaN     NaN     NaN
1 2020-01-03    SPY  321.160004  323.640015  321.100006  322.410004  77709700 ␣
↪294.640045 -0.007601     NaN     NaN
2 2020-01-06    SPY  320.489990  323.730011  320.359985  323.640015  55653900 ␣
↪295.764160  0.003808     NaN     NaN
```

```
3 2020-01-07    SPY  323.019989  323.540009  322.239990  322.730011  40496400 ␣
↪294.932526 -0.002816     NaN     NaN
4 2020-01-08    SPY  322.940002  325.779999  322.670013  324.450012  68296000 ␣
↪296.504425  0.005315     NaN     NaN
```

```
        date        r
0 2020-01-02  0.01495
1 2020-01-03  0.01473
2 2020-01-06  0.01488
3 2020-01-07  0.01500
4 2020-01-08  0.01493
```

[6]:
```python
print("Episode columns:")
print(episodes.columns.tolist())

print("Price columns:")
print(prices.columns.tolist())

print("Risk-free columns:")
print(rf.columns.tolist())
```

```
Episode columns:
['episode_id', 'regime', 'ticker', 'cp_flag', 'moneyness', 'inception_date',
'S0', 'strike', 'expiration', 'T_days', 'T_years', 'iv_inception_vix',
'iv_inception_vix3m', 'r_inception', 'div_amount_in_window', 'div_ex_dates',
'pv_div_inception', 'q_inception']
Price columns:
['date', 'ticker', 'open', 'high', 'low', 'close', 'volume', 'adj_close',
'logret', 'rv_20d', 'rv_60d']
Risk-free columns:
['date', 'r']
```

1.3 3. Prepare daily modeling data

Use the traded SPY `close` price for hedge marking, not the adjusted close. We merge risk-free rates onto trading days only, so SPY prices are not forward-filled into non-trading days.

[7]:
```python
# Ensure datetime format
prices = prices.copy()
rf = rf.copy()
prices["date"] = pd.to_datetime(prices["date"])
rf["date"] = pd.to_datetime(rf["date"])

data = prices.merge(rf, on="date", how="left")

# Risk-free rates can be filled across available trading rows; do not fill SPY␣
↪prices into non-trading days.
data["r"] = data["r"].ffill().bfill()
```

```
PRICE_COL = "close"
RF_COL = "r"

required_data_cols = ["date", PRICE_COL, RF_COL, "rv_20d", "rv_60d"]
missing_data_cols = [c for c in required_data_cols if c not in data.columns]
if missing_data_cols:
    raise KeyError(f"Missing required columns from merged data:␣
 ↪{missing_data_cols}")

display(data.head())
```

```
        date ticker        open        high         low       close    volume  ␣
 ↪adj_close    logret  rv_20d  rv_60d  \
0 2020-01-02    SPY  323.540009  324.890015  322.529999  324.869995  59151200 ␣
 ↪296.888123       NaN     NaN     NaN
1 2020-01-03    SPY  321.160004  323.640015  321.100006  322.410004  77709700 ␣
 ↪294.640045 -0.007601     NaN     NaN
2 2020-01-06    SPY  320.489990  323.730011  320.359985  323.640015  55653900 ␣
 ↪295.764160  0.003808     NaN     NaN
3 2020-01-07    SPY  323.019989  323.540009  322.239990  322.730011  40496400 ␣
 ↪294.932526 -0.002816     NaN     NaN
4 2020-01-08    SPY  322.940002  325.779999  322.670013  324.450012  68296000 ␣
 ↪296.504425  0.005315     NaN     NaN

         r
0  0.01495
1  0.01473
2  0.01488
3  0.01500
4  0.01493
```

1.4 4. Black–Scholes functions

The formulas include a continuous dividend-yield proxy `q`. The functions are vector-safe and include basic numerical protection near maturity or near-zero volatility.

[8]:
```
def bs_d1(S, K, r, q, sigma, tau):
    """Black-Scholes d1 with numerical safeguards."""
    S = np.asarray(S, dtype=float)
    K = np.asarray(K, dtype=float)
    r = np.asarray(r, dtype=float)
    q = np.asarray(q, dtype=float)
    sigma = np.asarray(sigma, dtype=float)
    tau = np.asarray(tau, dtype=float)

    sigma = np.maximum(sigma, 1e-8)
    tau = np.maximum(tau, 1e-8)
```

```
    return (np.log(S / K) + (r - q + 0.5 * sigma**2) * tau) / (sigma * np.
↪sqrt(tau))


def bs_d2(d1, sigma, tau):
    sigma = np.asarray(sigma, dtype=float)
    tau = np.asarray(tau, dtype=float)
    sigma = np.maximum(sigma, 1e-8)
    tau = np.maximum(tau, 1e-8)
    return d1 - sigma * np.sqrt(tau)


def bs_price(S, K, r, q, sigma, tau, option_type="call"):
    """Black-Scholes price for European call or put with continuous dividend␣
↪yield."""
    d1 = bs_d1(S, K, r, q, sigma, tau)
    d2 = bs_d2(d1, sigma, tau)

    if option_type == "call":
        return S * np.exp(-q * tau) * norm.cdf(d1) - K * np.exp(-r * tau) *␣
↪norm.cdf(d2)
    elif option_type == "put":
        return K * np.exp(-r * tau) * norm.cdf(-d2) - S * np.exp(-q * tau) *␣
↪norm.cdf(-d1)
    else:
        raise ValueError("option_type must be 'call' or 'put'")


def bs_delta(S, K, r, q, sigma, tau, option_type="call"):
    """Black-Scholes delta for European call or put with continuous dividend␣
↪yield."""
    d1 = bs_d1(S, K, r, q, sigma, tau)

    if option_type == "call":
        delta = np.exp(-q * tau) * norm.cdf(d1)
    elif option_type == "put":
        delta = np.exp(-q * tau) * (norm.cdf(d1) - 1)
    else:
        raise ValueError("option_type must be 'call' or 'put'")

    return np.clip(delta, -1, 1)
```

1.5 5. Build Black–Scholes delta panel

This creates one row per option episode, per trading day, per volatility specification. Including both implied and realized volatility lets the project directly compare how volatility input choices

change theoretical deltas and downstream hedging performance.

[9]:
```
# Core columns in episodes.parquet
EPISODE_ID_COL = "episode_id"
REGIME_COL = "regime"
START_COL = "inception_date"
END_COL = "expiration"
STRIKE_COL = "strike"
CP_COL = "cp_flag"
MONEYNESS_COL = "moneyness"
S0_COL = "S0"
R0_COL = "r_inception"
T_COL = "T_years"
DIV_COL = "div_amount_in_window"

required_episode_cols = [
    EPISODE_ID_COL, REGIME_COL, START_COL, END_COL, STRIKE_COL,
    CP_COL, MONEYNESS_COL, S0_COL, R0_COL, T_COL
]
missing_episode_cols = [c for c in required_episode_cols if c not in episodes.
 ↪columns]
if missing_episode_cols:
    raise KeyError(f"Missing required columns from episodes:␣
 ↪{missing_episode_cols}")

#compare short-tenor implied vol, 3-month implied vol, and historical realized␣
 ↪vols.
VOL_METHODS = ["iv_inception_vix", "iv_inception_vix3m", "rv_20d", "rv_60d"]

missing_vol_cols = [c for c in ["iv_inception_vix", "iv_inception_vix3m"] if c␣
 ↪not in episodes.columns]
if missing_vol_cols:
    raise KeyError(f"Missing implied-vol columns from episodes:␣
 ↪{missing_vol_cols}")

results = []

for vol_method in VOL_METHODS:
    for _, row in episodes.iterrows():
        episode_id = row[EPISODE_ID_COL]
        regime = row[REGIME_COL]
        start = pd.to_datetime(row[START_COL])
        end = pd.to_datetime(row[END_COL])
        K = float(row[STRIKE_COL])
        S0 = float(row[S0_COL])
        option_type = "call" if row[CP_COL] == "C" else "put"
```

```
        # Continuous-dividend-yield proxy based on expected dividend cash␣
↪amount during the option window.
        # If the column is not available or invalid, fall back to q = 0.
        if DIV_COL in episodes.columns and pd.notna(row[DIV_COL]) and␣
↪row[T_COL] > 0 and S0 > 0:
            q = float(row[DIV_COL]) / S0 / float(row[T_COL])
            q_source = "div_amount_in_window / S0 / T_years"
        else:
            q = 0.0
            q_source = "fallback_zero"

        df = data[(data["date"] >= start) & (data["date"] <= end)].copy()
        if df.empty:
            print(f"Warning: no trading-day data for {episode_id} from {start.
↪date()} to {end.date()}")
            continue

        for _, obs in df.iterrows():
            tau = (end - obs["date"]).days / 365.0
            if tau <= 0:
                continue

            S = float(obs[PRICE_COL])
            r_rate = float(obs[RF_COL]) if pd.notna(obs[RF_COL]) else␣
↪float(row[R0_COL])

            if vol_method == "iv_inception_vix":
                sigma = row["iv_inception_vix"]
            elif vol_method == "iv_inception_vix3m":
                sigma = row["iv_inception_vix3m"]
            elif vol_method == "rv_20d":
                sigma = obs["rv_20d"]
            elif vol_method == "rv_60d":
                sigma = obs["rv_60d"]
            else:
                raise ValueError(f"Unknown vol_method: {vol_method}")

            if pd.isna(sigma) or sigma <= 0:
                continue

            sigma = float(sigma)
            price = float(bs_price(S, K, r_rate, q, sigma, tau, option_type))
            delta = float(bs_delta(S, K, r_rate, q, sigma, tau, option_type))

            # Keep delta in valid theoretical range and remove tiny␣
↪floating-point noise near zero.
            delta = float(np.clip(delta, -1.0, 1.0))
```

```
            if abs(delta) < 1e-10:
                delta = 0.0

            results.append({
                "episode_id": episode_id,
                "regime": regime,
                "date": obs["date"],
                "S": S,
                "K": K,
                "tau": tau,
                "r": r_rate,
                "q": q,
                "sigma": sigma,
                "bs_price": price,
                "delta": delta,
                "option_type": option_type,
                "cp_flag": row[CP_COL],
                "moneyness": row[MONEYNESS_COL],
                "vol_method": vol_method,
                "q_source": q_source,
                "inception_date": start,
                "expiration": end,
                "S0": S0
            })

bs_panel = pd.DataFrame(results).sort_values(["vol_method", "episode_id",␣
 ↪"date"]).reset_index(drop=True)

print("Black-Scholes panel shape:", bs_panel.shape)
display(bs_panel.head())
```

```
Black-Scholes panel shape: (2847, 19)

       episode_id regime       date           S      K       tau        r       ␣
 ↪ q   sigma   bs_price     delta  \
0  SPY_calm_C_ATM   calm 2021-03-01  389.579987  390.0  0.241096  0.00028  0.
 ↪013122  0.2335  16.989992  0.506741
1  SPY_calm_C_ATM   calm 2021-03-02  386.540009  390.0  0.238356  0.00035  0.
 ↪013122  0.2335  15.399808  0.479406
2  SPY_calm_C_ATM   calm 2021-03-03  381.420013  390.0  0.235616  0.00035  0.
 ↪013122  0.2335  12.971057  0.432680
3  SPY_calm_C_ATM   calm 2021-03-04  376.700012  390.0  0.232877  0.00028  0.
 ↪013122  0.2335  10.937231  0.389266
4  SPY_calm_C_ATM   calm 2021-03-05  383.630005  390.0  0.230137  0.00028  0.
 ↪013122  0.2335  13.758139  0.451908

  option_type cp_flag moneyness        vol_method                           ␣
 ↪q_source inception_date expiration  \
```

```
0        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
 ↪T_years     2021-03-01 2021-05-28
1        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
 ↪T_years     2021-03-01 2021-05-28
2        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
 ↪T_years     2021-03-01 2021-05-28
3        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
 ↪T_years     2021-03-01 2021-05-28
4        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
 ↪T_years     2021-03-01 2021-05-28

           S0
0  389.579987
1  389.579987
2  389.579987
3  389.579987
4  389.579987
```

1.6 6. Quality checks

These checks confirm the panel is ready for handoff: no missing model outputs, correct delta ranges, and sensible episode coverage.

[10]:
```
print("Shape:", bs_panel.shape)
print("Rows by volatility method:")
display(bs_panel.groupby("vol_method").size().rename("n_rows").to_frame())

print("Missing values:")
display(bs_panel.isna().sum().to_frame("missing_count"))

print("Delta range by option type and volatility method:")
display(
    bs_panel.groupby(["vol_method", "option_type"])["delta"]
    .describe()
)
```

```
Shape: (2847, 19)
Rows by volatility method:

                    n_rows
vol_method
iv_inception_vix       741
iv_inception_vix3m     741
rv_20d                 741
rv_60d                 624

Missing values:

                missing_count
episode_id                  0
```

```
regime                      0
date                        0
S                           0
K                           0
tau                         0
r                           0
q                           0
sigma                       0
bs_price                    0
delta                       0
option_type                 0
cp_flag                     0
moneyness                   0
vol_method                  0
q_source                    0
inception_date              0
expiration                  0
S0                          0

Delta range by option type and volatility method:

                                count      mean       std       min       25%  
 ↪   50%       75%           max
vol_method         option_type
iv_inception_vix   call         494.0  0.329537  0.294446  0.000000  0.009465  0.
 ↪302847  0.567085  9.999640e-01
                   put          247.0 -0.576541  0.325879 -0.999961 -0.963870 -0.
 ↪494723 -0.310319 -1.530698e-09
iv_inception_vix3m call         494.0  0.336787  0.283344  0.000000  0.029327  0.
 ↪326483  0.562621  9.999639e-01
                   put          247.0 -0.576027  0.312627 -0.999961 -0.932776 -0.
 ↪492196 -0.327421 -1.166364e-07
rv_20d             call         494.0  0.368143  0.299337  0.000000  0.127474  0.
 ↪272003  0.585284  9.999641e-01
                   put          247.0 -0.513411  0.315327 -0.999961 -0.798464 -0.
 ↪502336 -0.267798  0.000000e+00
rv_60d             call         416.0  0.359436  0.314924  0.000000  0.077412  0.
 ↪253607  0.596761  9.999641e-01
                   put          208.0 -0.512943  0.331967 -0.999961 -0.873176 -0.
 ↪496809 -0.227109  0.000000e+00
```

[11]:
```python
# Calls should be between 0 and 1; puts should be between -1 and 0.
bad_calls = bs_panel[
    (bs_panel["option_type"] == "call") &
    ((bs_panel["delta"] < -1e-12) | (bs_panel["delta"] > 1 + 1e-12))
]

bad_puts = bs_panel[
```

```
    (bs_panel["option_type"] == "put") &
    ((bs_panel["delta"] < -1 - 1e-12) | (bs_panel["delta"] > 1e-12))
]

print("Bad call rows:", len(bad_calls))
print("Bad put rows:", len(bad_puts))

if len(bad_calls) > 0:
    display(bad_calls.head())
if len(bad_puts) > 0:
    display(bad_puts.head())
```

```
Bad call rows: 0
Bad put rows: 0
```

[12]:
```
episode_summary = bs_panel.groupby(["vol_method", "episode_id"]).agg(
    n_days=("date", "count"),
    first_date=("date", "min"),
    last_date=("date", "max"),
    min_tau=("tau", "min"),
    max_tau=("tau", "max"),
    min_delta=("delta", "min"),
    max_delta=("delta", "max"),
    first_price=("bs_price", "first"),
    last_price=("bs_price", "last"),
    mean_sigma=("sigma", "mean")
)

display(episode_summary)
```

```
                                      n_days first_date  last_date   min_tau  ␣
↪max_tau     min_delta     max_delta  \
vol_method          episode_id
iv_inception_vix    SPY_calm_C_ATM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096  3.892658e-01  9.999640e-01
                    SPY_calm_C_OTM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096  2.411383e-01  9.791005e-01
                    SPY_calm_P_ATM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096 -6.076831e-01 -1.530698e-09
                    SPY_highvol_C_ATM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315  0.000000e+00  5.430961e-01
                    SPY_highvol_C_OTM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315  0.000000e+00  3.430716e-01
                    SPY_highvol_P_ATM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315 -9.999612e-01 -4.534977e-01
                    SPY_recent_C_ATM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616  3.501367e-01  9.969460e-01
```

```
                    SPY_recent_C_OTM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616  4.385506e-02  4.079310e-01
                    SPY_recent_P_ATM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616 -6.486290e-01 -2.985381e-03
                    SPY_stress_C_ATM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096  0.000000e+00  7.155668e-01
                    SPY_stress_C_OTM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096  0.000000e+00  4.875876e-01
                    SPY_stress_P_ATM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096 -9.999517e-01 -2.809612e-01
iv_inception_vix3m SPY_calm_C_ATM          63 2021-03-01 2021-05-27  0.002740  0.
↪241096  4.089553e-01  9.999639e-01
                    SPY_calm_C_OTM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096  2.750481e-01  9.621893e-01
                    SPY_calm_P_ATM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096 -5.879936e-01 -1.166364e-07
                    SPY_highvol_C_ATM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315  0.000000e+00  5.427009e-01
                    SPY_highvol_C_OTM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315  0.000000e+00  3.790008e-01
                    SPY_highvol_P_ATM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315 -9.999612e-01 -4.538929e-01
                    SPY_recent_C_ATM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616  3.717501e-01  9.914866e-01
                    SPY_recent_C_OTM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616  6.954542e-02  4.233430e-01
                    SPY_recent_P_ATM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616 -6.270156e-01 -8.444754e-03
                    SPY_stress_C_ATM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096  0.000000e+00  7.149172e-01
                    SPY_stress_C_OTM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096  0.000000e+00  4.877524e-01
                    SPY_stress_P_ATM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096 -9.999517e-01 -2.816108e-01
rv_20d              SPY_calm_C_ATM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096  3.291647e-01  9.999641e-01
                    SPY_calm_C_OTM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096  1.478472e-01  9.995269e-01
                    SPY_calm_P_ATM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096 -6.677842e-01  0.000000e+00
                    SPY_highvol_C_ATM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315  0.000000e+00  5.430871e-01
                    SPY_highvol_C_OTM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315  0.000000e+00  3.451556e-01
                    SPY_highvol_P_ATM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315 -9.999612e-01 -4.535067e-01
```

```
                    SPY_recent_C_ATM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616  3.264192e-01  9.999313e-01
                    SPY_recent_C_OTM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616  4.919664e-03  3.903330e-01
                    SPY_recent_P_ATM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616 -6.723465e-01 -1.204497e-07
                    SPY_stress_C_ATM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096  4.855939e-09  7.699766e-01
                    SPY_stress_C_OTM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096  0.000000e+00  4.746858e-01
                    SPY_stress_P_ATM       62 2020-02-03 2020-04-30  0.002740  0.
↪241096 -9.999517e-01 -2.265515e-01
rv_60d              SPY_calm_C_ATM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096  3.084654e-01  9.999641e-01
                    SPY_calm_C_OTM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096  1.206087e-01  9.998357e-01
                    SPY_calm_P_ATM         63 2021-03-01 2021-05-27  0.002740  0.
↪241096 -6.884835e-01  0.000000e+00
                    SPY_highvol_C_ATM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315  0.000000e+00  5.426830e-01
                    SPY_highvol_C_OTM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315  0.000000e+00  3.594163e-01
                    SPY_highvol_P_ATM      62 2022-04-01 2022-06-30  0.002740  0.
↪249315 -9.999612e-01 -4.539108e-01
                    SPY_recent_C_ATM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616  2.956427e-01  9.996865e-01
                    SPY_recent_C_OTM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616  1.424341e-02  3.454599e-01
                    SPY_recent_P_ATM       60 2025-10-01 2025-12-24  0.005479  0.
↪235616 -7.031229e-01 -2.448863e-04
                    SPY_stress_C_ATM       23 2020-03-30 2020-04-30  0.002740  0.
↪087671  3.336636e-04  1.670404e-01
                    SPY_stress_C_OTM       23 2020-03-30 2020-04-30  0.002740  0.
↪087671  4.532381e-07  9.605920e-02
                    SPY_stress_P_ATM       23 2020-03-30 2020-04-30  0.002740  0.
↪087671 -9.996180e-01 -8.321387e-01

                                        first_price     last_price  mean_sigma
vol_method          episode_id
iv_inception_vix    SPY_calm_C_ATM       16.989992   2.927499e+01    0.233500
                    SPY_calm_C_OTM        9.735632   1.031462e+01    0.233500
                    SPY_calm_P_ATM       18.614213   1.259206e-09    0.233500
                    SPY_highvol_C_ATM    17.112695   7.195427e-72    0.196300
                    SPY_highvol_C_OTM     8.556925  2.171826e-114    0.196300
                    SPY_highvol_P_ATM    18.223564   7.574477e+01    0.196300
                    SPY_recent_C_ATM     23.277283   2.247027e+01    0.162900
```

```
                    SPY_recent_C_OTM      9.963393   3.260133e-01    0.162900
                    SPY_recent_P_ATM     18.772231   7.526795e-03    0.162900
                    SPY_stress_C_ATM     11.322877   4.208629e-32    0.179700
                    SPY_stress_C_OTM      5.340996   5.857322e-64    0.179700
                    SPY_stress_P_ATM     11.394132   3.353322e+01    0.179700
iv_inception_vix3m SPY_calm_C_ATM       19.598268   2.927499e+01    0.267800
                    SPY_calm_C_OTM       12.181216   1.036430e+01    0.267800
                    SPY_calm_P_ATM       21.222490   1.244372e-07    0.267800
                    SPY_highvol_C_ATM    21.228070   5.244276e-48    0.242100
                    SPY_highvol_C_OTM    12.322135   5.002977e-76    0.242100
                    SPY_highvol_P_ATM    22.338939   7.574477e+01    0.242100
                    SPY_recent_C_ATM     26.441336   2.248981e+01    0.187600
                    SPY_recent_C_OTM     12.827760   5.502741e-01    0.187600
                    SPY_recent_P_ATM     21.936284   2.706620e-02    0.187600
                    SPY_stress_C_ATM     11.367095   7.178977e-32    0.180400
                    SPY_stress_C_OTM      5.379983   1.754146e-63    0.180400
                    SPY_stress_P_ATM     11.438349   3.353322e+01    0.180400
rv_20d              SPY_calm_C_ATM       11.490678   2.927499e+01    0.139658
                    SPY_calm_C_OTM        4.869216   1.027535e+01    0.139658
                    SPY_calm_P_ATM       13.114899   4.692679e-23    0.139658
                    SPY_highvol_C_ATM    20.145499   5.405250e-31    0.270310
                    SPY_highvol_C_OTM    11.315094   9.713121e-49    0.270310
                    SPY_highvol_P_ATM    21.256368   7.574477e+01    0.270310
                    SPY_recent_C_ATM     10.602786   2.246275e+01    0.124080
                    SPY_recent_C_OTM      0.779960   6.858049e-03    0.124080
                    SPY_recent_P_ATM      6.097734   9.671521e-08    0.124080
                    SPY_stress_C_ATM      7.574237   4.414470e-09    0.506973
                    SPY_stress_C_OTM      2.269867   4.327233e-17    0.506973
                    SPY_stress_P_ATM      7.645491   3.353322e+01    0.506973
rv_60d              SPY_calm_C_ATM        9.952502   2.927499e+01    0.148683
                    SPY_calm_C_OTM        3.633256   1.027508e+01    0.148683
                    SPY_calm_P_ATM       11.576723   2.120096e-27    0.148683
                    SPY_highvol_C_ATM    18.742604   5.511565e-34    0.251279
                    SPY_highvol_C_OTM    10.026315   1.568494e-53    0.251279
                    SPY_highvol_P_ATM    19.853473   7.574477e+01    0.251279
                    SPY_recent_C_ATM     13.557498   2.246315e+01    0.113069
                    SPY_recent_C_OTM      2.310081   1.087350e-01    0.113069
                    SPY_recent_P_ATM      9.052446   4.060167e-04    0.113069
                    SPY_stress_C_ATM      2.258222   7.900477e-04    0.595565
                    SPY_stress_C_OTM      1.215666   7.925143e-07    0.595565
                    SPY_stress_P_ATM     65.008674   3.353401e+01    0.595565
```

1.7 7. Report resources

[13]:
```python
import matplotlib.pyplot as plt

FIG_DIR = DATA_PATH / "report_figures"
```

```
TABLE_DIR = DATA_PATH / "report_tables"
FIG_DIR.mkdir(exist_ok=True)
TABLE_DIR.mkdir(exist_ok=True)

vol_label_map = {
    "iv_inception_vix": "VIX-Implied Volatility",
    "iv_inception_vix3m": "VIX3M-Implied Volatility",
    "rv_20d": "20-Day Realized Volatility",
    "rv_60d": "60-Day Realized Volatility"
}

sample_id = "SPY_calm_C_ATM"
sample = bs_panel[bs_panel["episode_id"] == sample_id].copy()
sample_info = sample.iloc[0]

print("Figure directory:", FIG_DIR)
print("Table directory:", TABLE_DIR)
print("Sample episode used for report figures:", sample_id)
```

```
Figure directory:
C:\Users\elain\Downloads\5010-Project\data\output\report_figures
Table directory: C:\Users\elain\Downloads\5010-Project\data\output\report_tables
Sample episode used for report figures: SPY_calm_C_ATM
```

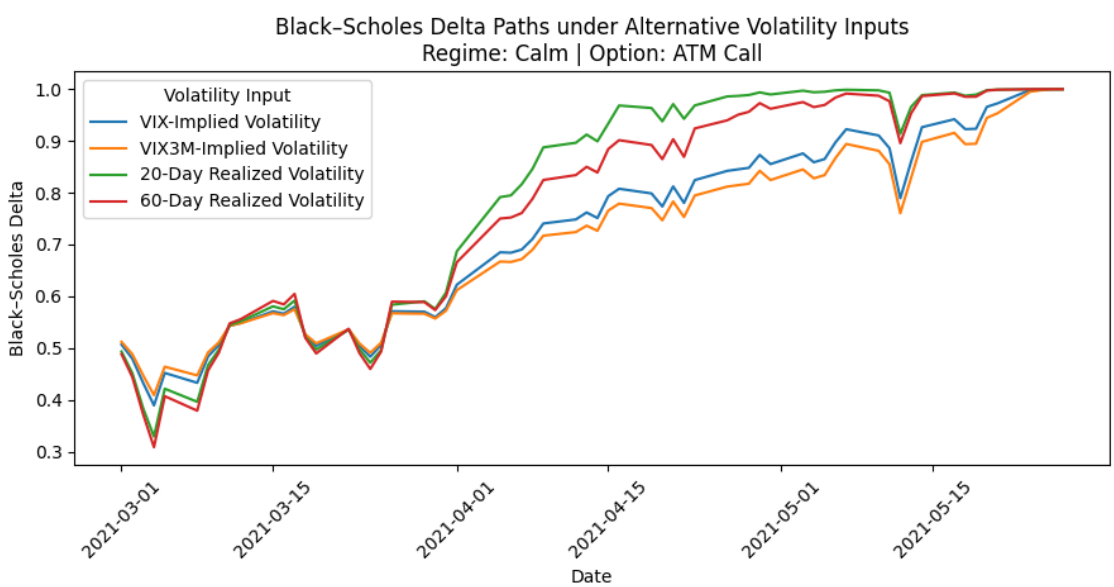
[14]:
```
vol_table = pd.DataFrame({
    "Volatility Input": [
        "VIX-Implied Volatility",
        "VIX3M-Implied Volatility",
        "20-Day Realized Volatility",
        "60-Day Realized Volatility"
    ],
    "Variable": [
        "iv_inception_vix",
        "iv_inception_vix3m",
        "rv_20d",
        "rv_60d"
    ],
    "Construction": [
        "30-day implied volatility fixed at inception",
        "3-month implied volatility fixed at inception",
        "Rolling realized volatility from daily returns",
        "Rolling realized volatility from daily returns"
    ],
    "Time Variation": [
        "Fixed within episode",
        "Fixed within episode",
        "Updated daily",
```

```
        "Updated daily"
    ]
})

display(vol_table)

vol_table.to_csv(TABLE_DIR / "table_volatility_inputs.csv", index=False)

fig, ax = plt.subplots(figsize=(14, 2.8))
ax.axis("off")
tbl = ax.table(
    cellText=vol_table.values,
    colLabels=vol_table.columns,
    loc="center",
    cellLoc="center"
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(10)
tbl.scale(1, 1.6)
plt.tight_layout()
plt.savefig(FIG_DIR / "table_volatility_inputs.png", dpi=300,␣
 ↪bbox_inches="tight")
plt.show()
plt.close()
```

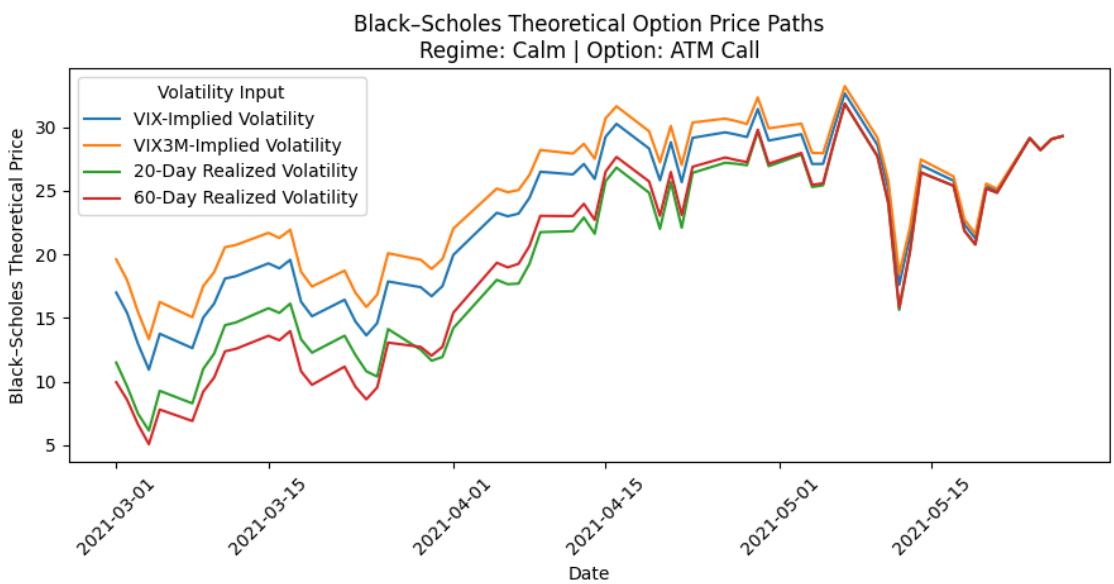
```
            Volatility Input            Variable                                ␣
 ↪    Construction  \
0      VIX-Implied Volatility    iv_inception_vix    30-day implied volatility␣
 ↪fixed at inception
1    VIX3M-Implied Volatility  iv_inception_vix3m   3-month implied volatility␣
 ↪fixed at inception
2  20-Day Realized Volatility              rv_20d  Rolling realized volatility␣
 ↪from daily returns
3  60-Day Realized Volatility              rv_60d  Rolling realized volatility␣
 ↪from daily returns

         Time Variation
0  Fixed within episode
1  Fixed within episode
2         Updated daily
3         Updated daily
```

Volatility Input	Variable	Construction	Time Variation
VIX-Implied Volatility	iv_inception_vix	30-day implied volatility fixed at inception	Fixed within episode
VIX3M-Implied Volatility	iv_inception_vix3m	3-month implied volatility fixed at inception	Fixed within episode
20-Day Realized Volatility	rv_20d	Rolling realized volatility from daily returns	Updated daily
60-Day Realized Volatility	rv_60d	Rolling realized volatility from daily returns	Updated daily

[15]:
```
formal_title = (
    "Black-Scholes Delta Paths under Alternative Volatility Inputs\n"
    f"Regime: {sample_info['regime'].title()} | "
    f"Option: {sample_info['moneyness']} {sample_info['option_type'].title()}"
)

plt.figure(figsize=(9, 4.8))
for method in sample["vol_method"].unique():
    sub = sample[sample["vol_method"] == method]
    plt.plot(sub["date"], sub["delta"], label=vol_label_map.get(method, method))

plt.title(formal_title)
plt.xlabel("Date")
plt.ylabel("Black-Scholes Delta")
plt.legend(title="Volatility Input")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(FIG_DIR / "fig_delta_paths.png", dpi=300, bbox_inches="tight")
plt.show()
plt.close()
```



[16]:
```
formal_title = (
    "Black-Scholes Theoretical Option Price Paths\n"
    f"Regime: {sample_info['regime'].title()} | "
    f"Option: {sample_info['moneyness']} {sample_info['option_type'].title()}"
)

plt.figure(figsize=(9, 4.8))
for method in sample["vol_method"].unique():
    sub = sample[sample["vol_method"] == method]
    plt.plot(sub["date"], sub["bs_price"], label=vol_label_map.get(method,␣
 ↪method))

plt.title(formal_title)
plt.xlabel("Date")
plt.ylabel("Black-Scholes Theoretical Price")
plt.legend(title="Volatility Input")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(FIG_DIR / "fig_price_paths.png", dpi=300, bbox_inches="tight")
plt.show()
plt.close()
```



[17]:
```
diagnostics_table = (
    bs_panel.groupby("vol_method")
    .agg(
```

```
        Obs=("delta", "count"),
        MeanSigma=("sigma", "mean"),
        MeanDelta=("delta", "mean"),
        MinDelta=("delta", "min"),
        MaxDelta=("delta", "max")
    )
    .reset_index()
)

label_map_short = {
    "iv_inception_vix": "VIX IV",
    "iv_inception_vix3m": "VIX3M IV",
    "rv_20d": "RV 20d",
    "rv_60d": "RV 60d"
}

diagnostics_table["vol_method"] = diagnostics_table["vol_method"].
 ↪map(label_map_short)
diagnostics_table = diagnostics_table.rename(columns={"vol_method": "Vol␣
 ↪Input"})

for col in ["MeanSigma", "MeanDelta", "MinDelta", "MaxDelta"]:
    diagnostics_table[col] = diagnostics_table[col].round(4)

diagnostics_table
fig, ax = plt.subplots(figsize=(9, 2.4))
ax.axis("off")

tbl = ax.table(
    cellText=diagnostics_table.values,
    colLabels=diagnostics_table.columns,
    loc="center",
    cellLoc="center"
)

tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
tbl.scale(1.05, 1.65)

plt.tight_layout()
plt.savefig(FIG_DIR / "table_model_diagnostics.png", dpi=300,␣
 ↪bbox_inches="tight")
plt.show()
```

Vol Input	Obs	MeanSigma	MeanDelta	MinDelta	MaxDelta
VIX IV	741	0.1935	0.0275	-1.0	1.0
VIX3M IV	741	0.2199	0.0325	-1.0	1.0
RV 20d	741	0.2609	0.0743	-1.0	1.0
RV 60d	624	0.2184	0.0686	-1.0	1.0

[18]:
```
report_summary = (
    bs_panel.groupby(["regime", "moneyness", "option_type", "vol_method"])
    .agg(
        mean_sigma=("sigma", "mean"),
        mean_delta=("delta", "mean"),
        sd_delta=("delta", "std"),
        mean_bs_price=("bs_price", "mean"),
        n_obs=("delta", "count")
    )
    .reset_index()
)

report_summary["vol_method_label"] = report_summary["vol_method"].
 ↪map(vol_label_map)
report_summary.to_csv(TABLE_DIR / "bs_delta_summary.csv", index=False)

display(report_summary.head(20))
```

```
     regime moneyness option_type          vol_method  mean_sigma  mean_delta ␣
 ↪sd_delta  mean_bs_price  n_obs  \
0      calm       ATM        call    iv_inception_vix    0.233500    0.723634  0.
 ↪180592      22.775460     63
1      calm       ATM        call  iv_inception_vix3m    0.267800    0.709678  0.
 ↪168839      24.255942     63
2      calm       ATM        call              rv_20d    0.139658    0.787731  0.
 ↪226650      19.800164     63
3      calm       ATM        call              rv_60d    0.148683    0.766657  0.
 ↪218295      19.565579     63
4      calm       ATM         put    iv_inception_vix    0.233500   -0.274767  0.
 ↪179677       7.894279     63
5      calm       ATM         put  iv_inception_vix3m    0.267800   -0.288724  0.
 ↪167923       9.374761     63
6      calm       ATM         put              rv_20d    0.139658   -0.210670  0.
 ↪225783       4.918983     63
7      calm       ATM         put              rv_60d    0.148683   -0.231745  0.
 ↪217405       4.684398     63
```

```
8      calm       OTM        call    iv_inception_vix    0.233500    0.514911  0.
↪166723      11.378959     63
9      calm       OTM        call  iv_inception_vix3m    0.267800    0.518135  0.
↪148632      13.075376     63
10     calm       OTM        call              rv_20d    0.139658    0.534474  0.
↪238458       7.138312     63
11     calm       OTM        call              rv_60d    0.148683    0.508809  0.
↪235664       7.477061     63
12  highvol       ATM        call    iv_inception_vix    0.196300    0.142903  0.
↪172513       3.578917     62
13  highvol       ATM        call  iv_inception_vix3m    0.242100    0.170132  0.
↪174308       5.053613     62
14  highvol       ATM        call              rv_20d    0.270310    0.185365  0.
↪153122       4.586958     62
15  highvol       ATM        call              rv_60d    0.251279    0.172608  0.
↪166633       4.625823     62
16  highvol       ATM         put    iv_inception_vix    0.196300   -0.855329  0.
↪173407      47.174011     62
17  highvol       ATM         put  iv_inception_vix3m    0.242100   -0.828099  0.
↪175238      48.648707     62
18  highvol       ATM         put              rv_20d    0.270310   -0.812866  0.
↪154071      48.182052     62
19  highvol       ATM         put              rv_60d    0.251279   -0.825623  0.
↪167565      48.220917     62

              vol_method_label
0       VIX-Implied Volatility
1     VIX3M-Implied Volatility
2   20-Day Realized Volatility
3   60-Day Realized Volatility
4       VIX-Implied Volatility
5     VIX3M-Implied Volatility
6   20-Day Realized Volatility
7   60-Day Realized Volatility
8       VIX-Implied Volatility
9     VIX3M-Implied Volatility
10  20-Day Realized Volatility
11  60-Day Realized Volatility
12      VIX-Implied Volatility
13    VIX3M-Implied Volatility
14  20-Day Realized Volatility
15  60-Day Realized Volatility
16      VIX-Implied Volatility
17    VIX3M-Implied Volatility
18  20-Day Realized Volatility
19  60-Day Realized Volatility
```

1.8 8. Dynamic delta hedging simulation

This section implements **Job C**: dynamic delta hedging, rebalancing logic, hedge-portfolio P&L tracking, and performance-series output.

The simulation uses a one-option **short option + delta hedge** convention:

- At inception, sell one model-priced option and hold `delta` shares of SPY.
- Between rebalancing dates, the stock position is held fixed and the cash account accrues at the risk-free rate.
- On rebalancing dates, the stock position is adjusted to the current Black–Scholes delta.
- At expiration, the remaining stock position is liquidated and the option payoff is settled.
- The terminal hedging P&L is the remaining cash after stock liquidation and option settlement.

A value near zero means the discrete hedge replicated the option payoff well. Large positive or negative values indicate hedging error from discrete rebalancing, volatility-input mismatch, realized price movement, and transaction costs.

[ ]:
```
def option_payoff(S, K, option_type):
    """Terminal payoff for one European option contract on one share."""
    if option_type == "call":
        return max(float(S) - float(K), 0.0)
    if option_type == "put":
        return max(float(K) - float(S), 0.0)
    raise ValueError("option_type must be 'call' or 'put'")


def accrue_cash(cash, annual_rate, start_date, end_date):
    """Accrue the cash account between two dates using continuous compounding.
 ↪"""
    dt = max((pd.to_datetime(end_date) - pd.to_datetime(start_date)).days, 0) /␣
 ↪365.0
    return float(cash) * np.exp(float(annual_rate) * dt)


def get_expiration_price(market_data, expiration_date, price_col=PRICE_COL):
    """
    Return the stock price used for option settlement.
    Prefer the exact expiration trading day; otherwise use the last available␣
 ↪trading day before expiration.
    """
    expiration_date = pd.to_datetime(expiration_date)
    md = market_data.copy()
    md["date"] = pd.to_datetime(md["date"])

    exact = md.loc[md["date"] == expiration_date, price_col]
    if len(exact) > 0:
        return float(exact.iloc[-1]), expiration_date, "exact_expiration_close"
```

```
    prior = md.loc[md["date"] < expiration_date, ["date", price_col]].
↪sort_values("date")
    if prior.empty:
        raise ValueError(f"No market price available on or before expiration␣
↪{expiration_date.date()}")

    last = prior.iloc[-1]
    return float(last[price_col]), pd.to_datetime(last["date"]),␣
↪"last_available_close_before_expiration"


def simulate_delta_hedge(
    episode_panel,
    market_data,
    rebalance_step=1,
    rebalance_label="daily",
    transaction_cost_bps=0.0,
    price_col=PRICE_COL
):
    """
    Simulate a discrete dynamic delta hedge for one episode and one volatility␣
↪method.

    Convention:
    - short one option
    - hold +delta shares to hedge the short option
    - cash account starts with the model option premium
    """
    sub = episode_panel.sort_values("date").reset_index(drop=True).copy()
    if sub.empty:
        return pd.DataFrame()

    first = sub.iloc[0]
    episode_id = first["episode_id"]
    vol_method = first["vol_method"]
    option_type = first["option_type"]
    K = float(first["K"])
    expiration = pd.to_datetime(first["expiration"])

    # Initial short-option premium finances the hedge.
    cash = float(first["bs_price"])
    shares = 0.0
    prev_date = pd.to_datetime(first["date"])
    prev_r = float(first["r"])

    rows = []
    total_transaction_cost = 0.0
```

```
    total_turnover_dollars = 0.0
    n_rebalances = 0

    for i, row in sub.iterrows():
        current_date = pd.to_datetime(row["date"])
        S = float(row["S"])
        r_now = float(row["r"])

        # Accrue cash from the previous trading date to the current trading␣
↪date.
        if i > 0:
            cash = accrue_cash(cash, prev_r, prev_date, current_date)

        # Rebalance at inception and then every rebalance_step trading rows.
        is_rebalance_date = (i == 0) or (i % int(rebalance_step) == 0)

        trade_shares = 0.0
        transaction_cost = 0.0
        target_delta = float(row["delta"])

        if is_rebalance_date:
            target_shares = target_delta
            trade_shares = target_shares - shares
            turnover_dollars = abs(trade_shares) * S
            transaction_cost = turnover_dollars * float(transaction_cost_bps) /␣
↪10000.0

            # Buying shares reduces cash; selling shares increases cash. Costs␣
↪always reduce cash.
            cash -= trade_shares * S + transaction_cost
            shares = target_shares

            total_transaction_cost += transaction_cost
            total_turnover_dollars += turnover_dollars
            n_rebalances += 1

        stock_value = shares * S
        hedge_asset_value = cash + stock_value
        option_mark_value = float(row["bs_price"])

        rows.append({
            "episode_id": episode_id,
            "regime": row["regime"],
            "moneyness": row["moneyness"],
            "option_type": option_type,
            "vol_method": vol_method,
            "rebalance_frequency": rebalance_label,
```

```
            "rebalance_step_trading_days": int(rebalance_step),
            "transaction_cost_bps": float(transaction_cost_bps),
            "date": current_date,
            "row_type": "hedge_mark",
            "S": S,
            "K": K,
            "r": r_now,
            "sigma": float(row["sigma"]),
            "tau": float(row["tau"]),
            "option_value": option_mark_value,
            "target_delta": target_delta,
            "stock_shares": shares,
            "trade_shares": trade_shares,
            "is_rebalance_date": bool(is_rebalance_date),
            "cash_account": cash,
            "stock_value": stock_value,
            "hedge_asset_value": hedge_asset_value,
            "mark_to_market_hedging_error": hedge_asset_value -␣
↪option_mark_value,
            "period_transaction_cost": transaction_cost,
            "cumulative_transaction_cost": total_transaction_cost,
            "cumulative_turnover_dollars": total_turnover_dollars,
            "n_rebalances_so_far": n_rebalances,
            "expiration": expiration
        })

        prev_date = current_date
        prev_r = r_now

    # Expiration closeout: accrue cash, liquidate shares, and settle payoff.
    S_exp, settlement_date, settlement_source =␣
↪get_expiration_price(market_data, expiration, price_col=price_col)
    cash_before_settlement = accrue_cash(cash, prev_r, prev_date,␣
↪settlement_date)

    stock_liquidation_value = shares * S_exp
    closeout_turnover = abs(shares) * S_exp
    closeout_transaction_cost = closeout_turnover * float(transaction_cost_bps)␣
↪/ 10000.0
    payoff = option_payoff(S_exp, K, option_type)

    terminal_cash = cash_before_settlement + stock_liquidation_value -␣
↪closeout_transaction_cost - payoff

    total_transaction_cost += closeout_transaction_cost
    total_turnover_dollars += closeout_turnover
```

```
    rows.append({
        "episode_id": episode_id,
        "regime": first["regime"],
        "moneyness": first["moneyness"],
        "option_type": option_type,
        "vol_method": vol_method,
        "rebalance_frequency": rebalance_label,
        "rebalance_step_trading_days": int(rebalance_step),
        "transaction_cost_bps": float(transaction_cost_bps),
        "date": settlement_date,
        "row_type": "expiration_settlement",
        "S": S_exp,
        "K": K,
        "r": prev_r,
        "sigma": np.nan,
        "tau": 0.0,
        "option_value": payoff,
        "target_delta": 0.0,
        "stock_shares": 0.0,
        "trade_shares": -shares,
        "is_rebalance_date": True,
        "cash_account": terminal_cash,
        "stock_value": 0.0,
        "hedge_asset_value": terminal_cash,
        "mark_to_market_hedging_error": terminal_cash,
        "period_transaction_cost": closeout_transaction_cost,
        "cumulative_transaction_cost": total_transaction_cost,
        "cumulative_turnover_dollars": total_turnover_dollars,
        "n_rebalances_so_far": n_rebalances,
        "expiration": expiration,
        "settlement_price_source": settlement_source,
        "terminal_payoff": payoff,
        "terminal_hedging_pnl": terminal_cash
    })

    return pd.DataFrame(rows)


# Rebalancing designs for the project comparison.
# These are trading-day based frequencies. The daily case is the baseline.
REBALANCE_SPECS = {
    "daily": 1,
    "every_2_days": 2,
    "weekly": 5
}

# 0 bps = frictionless BS-style benchmark.
```

```python
# 1 and 5 bps allow the report to discuss how small trading frictions change␣
↪hedging performance.
TRANSACTION_COST_BPS_LIST = [0.0, 1.0, 5.0]

hedge_runs = []
group_cols = ["episode_id", "vol_method"]

for (episode_id, vol_method), sub_panel in bs_panel.groupby(group_cols):
    for freq_label, step in REBALANCE_SPECS.items():
        for cost_bps in TRANSACTION_COST_BPS_LIST:
            sim = simulate_delta_hedge(
                episode_panel=sub_panel,
                market_data=data,
                rebalance_step=step,
                rebalance_label=freq_label,
                transaction_cost_bps=cost_bps,
                price_col=PRICE_COL
            )
            hedge_runs.append(sim)

hedge_performance_series = (
    pd.concat(hedge_runs, ignore_index=True)
    .sort_values(["transaction_cost_bps", "rebalance_frequency", "vol_method",␣
↪"episode_id", "date", "row_type"])
    .reset_index(drop=True)
)

print("Hedging performance series shape:", hedge_performance_series.shape)
display(hedge_performance_series.head())
display(hedge_performance_series.tail())
```

```
Hedging performance series shape: (26055, 32)

       episode_id regime moneyness option_type        vol_method␣
↪rebalance_frequency  rebalance_step_trading_days  \
0  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                            1
1  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                            1
2  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                            1
3  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                            1
4  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                            1

   transaction_cost_bps       date    row_type           S      K        r  ␣
↪sigma       tau  option_value  \
```

```
0                    0.0 2021-03-01  hedge_mark  389.579987  390.0  0.00028  0.
↪2335  0.241096     16.989992
1                    0.0 2021-03-02  hedge_mark  386.540009  390.0  0.00035  0.
↪2335  0.238356     15.399808
2                    0.0 2021-03-03  hedge_mark  381.420013  390.0  0.00035  0.
↪2335  0.235616     12.971057
3                    0.0 2021-03-04  hedge_mark  376.700012  390.0  0.00028  0.
↪2335  0.232877     10.937231
4                    0.0 2021-03-05  hedge_mark  383.630005  390.0  0.00028  0.
↪2335  0.230137     13.758139

   target_delta  stock_shares  trade_shares  is_rebalance_date  cash_account ␣
↪stock_value  hedge_asset_value  \
0      0.506741      0.506741      0.506741               True   -180.426121  ␣
↪197.416112          16.989992
1      0.479406      0.479406     -0.027335               True   -169.860213  ␣
↪185.309585          15.449372
2      0.432680      0.432680     -0.046726               True   -152.038116  ␣
↪165.032769          12.994653
3      0.389266      0.389266     -0.043414               True   -135.684163  ␣
↪146.636421          10.952258
4      0.451908      0.451908      0.062642               True   -159.715719  ␣
↪173.365481          13.649763

   mark_to_market_hedging_error  period_transaction_cost ␣
↪cumulative_transaction_cost  cumulative_turnover_dollars  \
0                      0.000000                      0.0                        ␣
↪ 0.0                   197.416112
1                      0.049564                      0.0                        ␣
↪ 0.0                   207.982158
2                      0.023596                      0.0                        ␣
↪ 0.0                   225.804419
3                      0.015026                      0.0                        ␣
↪ 0.0                   242.158517
4                     -0.108376                      0.0                        ␣
↪ 0.0                   266.189968

   n_rebalances_so_far expiration settlement_price_source  terminal_payoff ␣
↪terminal_hedging_pnl
0                    1 2021-05-28                      NaN              NaN      ␣
↪            NaN
1                    2 2021-05-28                      NaN              NaN      ␣
↪            NaN
2                    3 2021-05-28                      NaN              NaN      ␣
↪            NaN
```

```
3                    4 2021-05-28                      NaN              NaN
↪            NaN
4                    5 2021-05-28                      NaN              NaN
↪            NaN

            episode_id  regime moneyness option_type vol_method␣
↪rebalance_frequency  rebalance_step_trading_days  \
26050  SPY_stress_P_ATM  stress       ATM         put     rv_60d
↪weekly                             5
26051  SPY_stress_P_ATM  stress       ATM         put     rv_60d
↪weekly                             5
26052  SPY_stress_P_ATM  stress       ATM         put     rv_60d
↪weekly                             5
26053  SPY_stress_P_ATM  stress       ATM         put     rv_60d
↪weekly                             5
26054  SPY_stress_P_ATM  stress       ATM         put     rv_60d
↪weekly                             5

       transaction_cost_bps       date               row_type           S      K␣
↪       r     sigma       tau  \
26050                   5.0 2020-04-27             hedge_mark  287.049988  324.0␣
↪ 0.00093  0.609903  0.010959
26051                   5.0 2020-04-28             hedge_mark  285.730011  324.0␣
↪ 0.00085  0.609002  0.008219
26052                   5.0 2020-04-29             hedge_mark  293.209991  324.0␣
↪ 0.00090  0.611383  0.005479
26053                   5.0 2020-04-30             hedge_mark  290.480011  324.0␣
↪ 0.00090  0.610568  0.002740
26054                   5.0 2020-05-01  expiration_settlement  282.790009  324.0␣
↪ 0.00090       NaN  0.000000

       option_value  target_delta  stock_shares  trade_shares  is_rebalance_date␣
↪ cash_account  stock_value  \
26050     37.217657     -0.968893     -0.950786      0.000000              False␣
↪   313.747089  -272.923149
26051     38.374484     -0.987680     -0.987680     -0.036894               True␣
↪   324.284320  -282.209835
26052     30.883271     -0.985490     -0.987680      0.000000              False␣
↪   324.285075  -289.597663
26053     33.534012     -0.999618     -0.987680      0.000000              False␣
↪   324.285874  -286.901315
26054     41.209991      0.000000      0.000000      0.987680               True␣
↪     3.630976     0.000000

       hedge_asset_value  mark_to_market_hedging_error  period_transaction_cost ␣
↪cumulative_transaction_cost  \
```

```
26050          40.823940                      3.606283                 0.000000 ␣
↪                   0.140947
26051          42.074485                      3.700001                 0.005271 ␣
↪                   0.146218
26052          34.687412                      3.804142                 0.000000 ␣
↪                   0.146218
26053          37.384559                      3.850548                 0.000000 ␣
↪                   0.146218
26054           3.630976                      3.630976                 0.139653 ␣
↪                   0.285871

       cumulative_turnover_dollars  n_rebalances_so_far expiration␣
↪settlement_price_source  terminal_payoff  \
26050                   281.893883                    4 2020-05-01              ␣
↪      NaN              NaN
26051                   292.435585                    5 2020-05-01              ␣
↪      NaN              NaN
26052                   292.435585                    5 2020-05-01              ␣
↪      NaN              NaN
26053                   292.435585                    5 2020-05-01              ␣
↪      NaN              NaN
26054                   571.741638                    5 2020-05-01 ␣
↪exact_expiration_close        41.209991

       terminal_hedging_pnl
26050                   NaN
26051                   NaN
26052                   NaN
26053                   NaN
26054              3.630976
```

1.9 9. Hedging performance summary and checks

This summary compresses the daily hedge paths into one row per episode × volatility method × rebalancing rule × transaction-cost assumption. The key outcome is `terminal_hedging_pnl`, the final replication error after stock liquidation and option payoff settlement.

[20]:
```
# Separate live hedge marks from terminal settlement rows.
hedge_marks = hedge_performance_series[hedge_performance_series["row_type"] ==␣
↪"hedge_mark"].copy()
terminal_rows = hedge_performance_series[hedge_performance_series["row_type"]␣
↪== "expiration_settlement"].copy()

hedge_tracking_summary = (
    hedge_marks
    .groupby(["episode_id", "regime", "moneyness", "option_type", "vol_method",
```

```
                "rebalance_frequency", "rebalance_step_trading_days",␣
↪"transaction_cost_bps"])
    .agg(
        n_marking_days=("date", "count"),
        mean_abs_mtm_error=("mark_to_market_hedging_error", lambda x: np.
↪mean(np.abs(x))),
        rmse_mtm_error=("mark_to_market_hedging_error", lambda x: np.sqrt(np.
↪mean(np.square(x)))),
        max_abs_mtm_error=("mark_to_market_hedging_error", lambda x: np.max(np.
↪abs(x))),
        avg_abs_delta_trade=("trade_shares", lambda x: np.mean(np.abs(x))),
        n_rebalance_days=("is_rebalance_date", "sum"),
        total_transaction_cost=("cumulative_transaction_cost", "max"),
        total_turnover_dollars=("cumulative_turnover_dollars", "max")
    )
    .reset_index()
)

terminal_summary = (
    terminal_rows
    .loc[:, ["episode_id", "vol_method", "rebalance_frequency",␣
↪"transaction_cost_bps",
             "date", "S", "option_value", "terminal_payoff",␣
↪"terminal_hedging_pnl",
             "settlement_price_source"]]
    .rename(columns={
        "date": "settlement_date",
        "S": "settlement_stock_price",
        "option_value": "settlement_option_payoff"
    })
)

hedge_summary = hedge_tracking_summary.merge(
    terminal_summary,
    on=["episode_id", "vol_method", "rebalance_frequency",␣
↪"transaction_cost_bps"],
    how="left"
)

hedge_summary["abs_terminal_hedging_pnl"] =␣
↪hedge_summary["terminal_hedging_pnl"].abs()

print("Hedge summary shape:", hedge_summary.shape)
display(
    hedge_summary
```

```
    .sort_values(["transaction_cost_bps", "rebalance_frequency",␣
 ↪"abs_terminal_hedging_pnl"])
    .head(12)
)

print("Average absolute terminal hedging error by rebalancing frequency and␣
 ↪transaction cost:")
display(
    hedge_summary
    .groupby(["transaction_cost_bps",␣
 ↪"rebalance_frequency"])["abs_terminal_hedging_pnl"]
    .mean()
    .unstack("rebalance_frequency")
    .round(4)
)

print("Average absolute terminal hedging error by volatility input:")
display(
    hedge_summary
    .groupby(["transaction_cost_bps", "vol_method"])["abs_terminal_hedging_pnl"]
    .mean()
    .unstack("vol_method")
    .round(4)
)
```

```
Hedge summary shape: (432, 23)

            episode_id   regime moneyness option_type          vol_method␣
 ↪rebalance_frequency  \
99       SPY_calm_P_ATM     calm       ATM         put              rv_60d       ␣
 ↪       daily
63       SPY_calm_C_OTM     calm       OTM        call              rv_60d       ␣
 ↪       daily
189  SPY_highvol_P_ATM  highvol       ATM         put  iv_inception_vix3m       ␣
 ↪       daily
54       SPY_calm_C_OTM     calm       OTM        call              rv_20d       ␣
 ↪       daily
162  SPY_highvol_C_OTM  highvol       OTM        call              rv_20d       ␣
 ↪       daily
270   SPY_recent_C_OTM   recent       OTM        call              rv_20d       ␣
 ↪       daily
153  SPY_highvol_C_OTM  highvol       OTM        call  iv_inception_vix3m       ␣
 ↪       daily
27       SPY_calm_C_ATM     calm       ATM        call              rv_60d       ␣
 ↪       daily
18       SPY_calm_C_ATM     calm       ATM        call              rv_20d       ␣
 ↪       daily
```

```
387   SPY_stress_C_OTM   stress       OTM        call              rv_60d
       daily
198  SPY_highvol_P_ATM  highvol       ATM         put              rv_20d
       daily
144  SPY_highvol_C_OTM  highvol       OTM        call    iv_inception_vix
       daily

     rebalance_step_trading_days  transaction_cost_bps  n_marking_days
mean_abs_mtm_error  rmse_mtm_error  \
99                             1                   0.0              63
 0.487140        0.636846
63                             1                   0.0              63
 0.678309        0.864928
189                            1                   0.0              62
 0.828659        0.912376
54                             1                   0.0              63
 1.979381        2.427215
162                            1                   0.0              62
 2.069116        2.472410
270                            1                   0.0              60
 2.921677        3.284882
153                            1                   0.0              62
 0.873490        0.945765
27                             1                   0.0              63
 0.952561        1.016162
18                             1                   0.0              63
 1.781113        1.904529
387                            1                   0.0              23
 0.652623        0.842109
198                            1                   0.0              62
 3.285809        3.715498
144                            1                   0.0              62
 1.246017        1.439556

     max_abs_mtm_error  avg_abs_delta_trade  n_rebalance_days
total_transaction_cost  total_turnover_dollars  \
99            1.332724             0.036124                63
 0.0              903.672951
63            1.915821             0.054986                63
 0.0             1409.850152
189           1.741310             0.035395                62
 0.0              936.515789
54            4.173871             0.061792                63
 0.0             1587.012600
162           5.154037             0.024528                62
 0.0              653.306017
```

```
270            5.362819             0.034386                 60                    ␣
↪ 0.0              1390.931362
153            1.237710             0.021823                 62                    ␣
↪ 0.0               584.841091
27             1.612192             0.035817                 63                    ␣
↪ 0.0               896.150843
18             2.652172             0.032962                 63                    ␣
↪ 0.0               823.479003
387            1.425279             0.017673                 23                    ␣
↪ 0.0               110.219559
198            6.653574             0.039126                 62                    ␣
↪ 0.0              1035.139326
144            1.832595             0.018844                 62                    ␣
↪ 0.0               509.159152

    settlement_date  settlement_stock_price  settlement_option_payoff ␣
↪terminal_payoff  terminal_hedging_pnl  \
99       2021-05-28              420.040009                  0.000000         0.
↪000000             -0.062937
63       2021-05-28              420.040009                 11.040009        11.
↪040009              0.359239
189      2022-07-01              381.239990                 71.760010        71.
↪760010             -0.427685
54       2021-05-28              420.040009                 11.040009        11.
↪040009              0.670057
162      2022-07-01              381.239990                  0.000000         0.
↪000000             -0.865055
270      2025-12-26              690.309998                  0.000000         0.
↪000000             -1.149374
153      2022-07-01              381.239990                  0.000000         0.
↪000000             -1.154723
27       2021-05-28              420.040009                 30.040009        30.
↪040009             -1.331433
18       2021-05-28              420.040009                 30.040009        30.
↪040009              1.359718
387      2020-05-01              282.790009                  0.000000         0.
↪000000              1.425280
198      2022-07-01              381.239990                 71.760010        71.
↪760010             -1.491209
144      2022-07-01              381.239990                  0.000000         0.
↪000000             -1.829710

    settlement_price_source  abs_terminal_hedging_pnl
99   exact_expiration_close                  0.062937
63   exact_expiration_close                  0.359239
189  exact_expiration_close                  0.427685
```

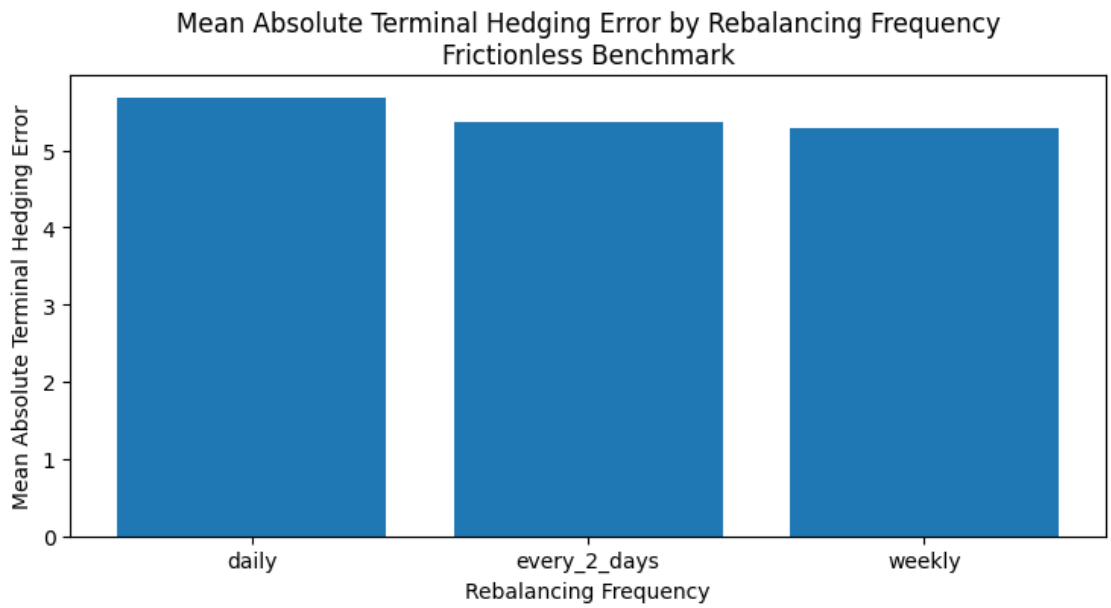
```
54   exact_expiration_close                  0.670057
162  exact_expiration_close                  0.865055
270  exact_expiration_close                  1.149374
153  exact_expiration_close                  1.154723
27   exact_expiration_close                  1.331433
18   exact_expiration_close                  1.359718
387  exact_expiration_close                  1.425280
198  exact_expiration_close                  1.491209
144  exact_expiration_close                  1.829710

Average absolute terminal hedging error by rebalancing frequency and transaction
cost:

rebalance_frequency    daily  every_2_days  weekly
transaction_cost_bps
0.0                   5.6880        5.3690  5.2885
1.0                   5.7077        5.3811  5.2802
5.0                   5.8231        5.4296  5.2505

Average absolute terminal hedging error by volatility input:

vol_method            iv_inception_vix  iv_inception_vix3m  rv_20d  rv_60d
transaction_cost_bps
0.0                             5.0375              6.4276  7.2127  3.1163
1.0                             4.9961              6.3897  7.2637  3.1757
5.0                             4.8308              6.2384  7.4863  3.4489
```

[21]:
```python
# Report-ready hedging figures and tables.
hedge_summary.to_csv(TABLE_DIR / "hedge_summary.csv", index=False)

frequency_table = (
    hedge_summary
    .groupby(["transaction_cost_bps", "rebalance_frequency"])
    .agg(
        MeanAbsTerminalError=("abs_terminal_hedging_pnl", "mean"),
        MedianAbsTerminalError=("abs_terminal_hedging_pnl", "median"),
        MeanTransactionCost=("total_transaction_cost", "mean"),
        MeanTurnover=("total_turnover_dollars", "mean")
    )
    .reset_index()
)
frequency_table.to_csv(TABLE_DIR / "hedge_rebalance_frequency_summary.csv",␣
 ↪index=False)
display(frequency_table)

# Figure 1: hedging error comparison by rebalancing frequency under␣
 ↪frictionless costs.
frictionless = hedge_summary[hedge_summary["transaction_cost_bps"] == 0.0].
 ↪copy()
```

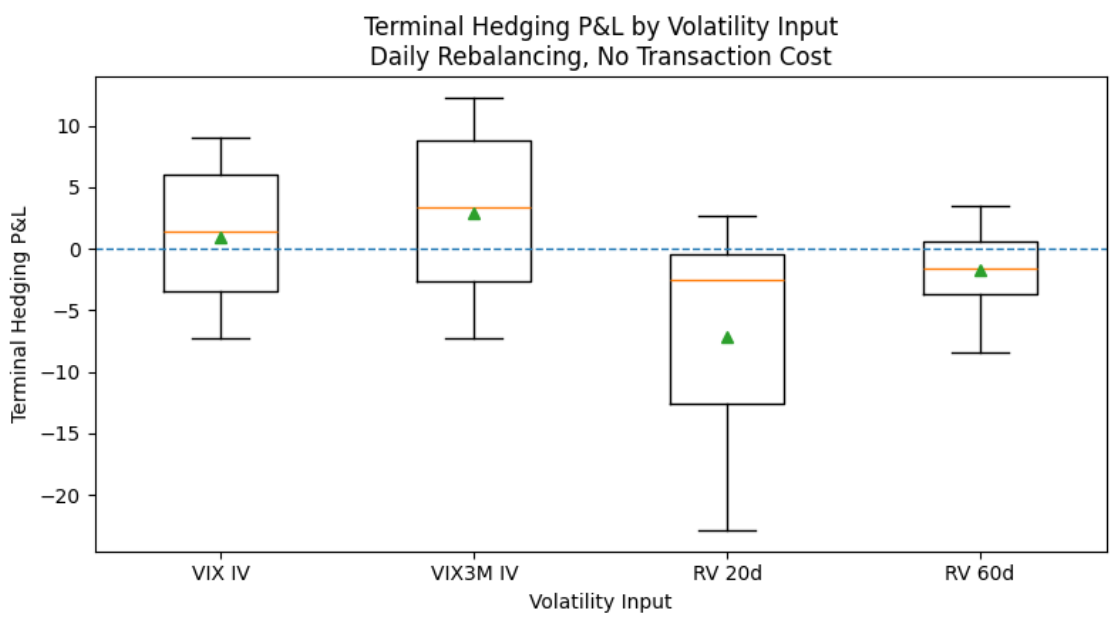
```
freq_order = ["daily", "every_2_days", "weekly"]
freq_plot = (
    frictionless
    .groupby("rebalance_frequency")["abs_terminal_hedging_pnl"]
    .mean()
    .reindex(freq_order)
)

plt.figure(figsize=(7.5, 4.2))
plt.bar(freq_plot.index, freq_plot.values)
plt.title("Mean Absolute Terminal Hedging Error by Rebalancing␣
 ↪Frequency\nFrictionless Benchmark")
plt.xlabel("Rebalancing Frequency")
plt.ylabel("Mean Absolute Terminal Hedging Error")
plt.tight_layout()
plt.savefig(FIG_DIR / "fig_hedge_error_by_rebalance_frequency.png", dpi=300,␣
 ↪bbox_inches="tight")
plt.show()
plt.close()

# Figure 2: volatility-input comparison under daily rebalancing and no␣
 ↪transaction cost.
daily_zero = hedge_summary[
    (hedge_summary["rebalance_frequency"] == "daily") &
    (hedge_summary["transaction_cost_bps"] == 0.0)
].copy()

vol_order = [m for m in VOL_METHODS if m in daily_zero["vol_method"].unique()]
vol_plot_data = [
    daily_zero.loc[daily_zero["vol_method"] == method, "terminal_hedging_pnl"].
 ↪dropna().values
    for method in vol_order
]
vol_labels = [label_map_short.get(method, method) for method in vol_order]

plt.figure(figsize=(8, 4.5))
plt.boxplot(vol_plot_data, labels=vol_labels, showmeans=True)
plt.axhline(0, linestyle="--", linewidth=1)
plt.title("Terminal Hedging P&L by Volatility Input\nDaily Rebalancing, No␣
 ↪Transaction Cost")
plt.xlabel("Volatility Input")
plt.ylabel("Terminal Hedging P&L")
plt.tight_layout()
plt.savefig(FIG_DIR / "fig_terminal_hedging_pnl_by_vol_input.png", dpi=300,␣
 ↪bbox_inches="tight")
plt.show()
```

```
plt.close()
```

```
   transaction_cost_bps rebalance_frequency  MeanAbsTerminalError ␣
↪MedianAbsTerminalError  MeanTransactionCost  \
0                   0.0               daily              5.688040                ␣
↪4.442981             0.000000
1                   0.0        every_2_days              5.368957                ␣
↪4.099208             0.000000
2                   0.0              weekly              5.288495                ␣
↪4.402892             0.000000
3                   1.0               daily              5.707652                ␣
↪4.346581             0.111678
4                   1.0        every_2_days              5.381081                ␣
↪4.070282             0.084897
5                   1.0              weekly              5.280188                ␣
↪4.422199             0.058388
6                   5.0               daily              5.823147                ␣
↪4.123830             0.558389
7                   5.0        every_2_days              5.429574                ␣
↪4.101394             0.424484
8                   5.0              weekly              5.250517                ␣
↪4.280477             0.291939

   MeanTurnover
0   1116.778392
1    848.968148
2    583.878995
3   1116.778392
4    848.968148
5    583.878995
6   1116.778392
7    848.968148
8    583.878995
```




```
C:\Users\elain\AppData\Local\Temp\ipykernel_70932\3299027378.py:52:
MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be
dropped in 3.11.
  plt.boxplot(vol_plot_data, labels=vol_labels, showmeans=True)
```



1.10 10. Save deliverables

[ ]:
```
output_parquet = DATA_PATH / "bs_delta_panel.parquet"
output_csv = DATA_PATH / "bs_delta_panel.csv"
hedge_series_parquet = DATA_PATH / "hedge_performance_series.parquet"
hedge_series_csv = DATA_PATH / "hedge_performance_series.csv"

episode_summary_csv = TABLE_DIR / "episode_summary.csv"
summary_csv = TABLE_DIR / "bs_delta_summary.csv"
hedge_summary_csv = TABLE_DIR / "hedge_summary.csv"
hedge_frequency_csv = TABLE_DIR / "hedge_rebalance_frequency_summary.csv"

readme_path = DATA_PATH / "bs_delta_panel_README.md"
hedge_readme_path = DATA_PATH / "hedge_performance_README.md"
manifest_path = DATA_PATH / "modeling_outputs_manifest.txt"

# Job B Model handoff files
bs_panel.to_parquet(output_parquet, index=False)
bs_panel.to_csv(output_csv, index=False)
episode_summary.to_csv(episode_summary_csv)
report_summary.to_csv(summary_csv, index=False)

# Job C hedging handoff files
hedge_performance_series.to_parquet(hedge_series_parquet, index=False)
hedge_performance_series.to_csv(hedge_series_csv, index=False)
hedge_summary.to_csv(hedge_summary_csv, index=False)
frequency_table.to_csv(hedge_frequency_csv, index=False)

readme_text = f"""# Black-Scholes Delta Panel

Generated by `black_scholes_model.ipynb`.

## Main handoff files
- `bs_delta_panel.parquet`
- `bs_delta_panel.csv`

## Unit of observation
One row = one option episode × one trading day × one volatility method.

## Key columns
- `episode_id`: option experiment identifier.
- `regime`: market regime label.
- `date`: trading date.
- `S`: SPY traded close price.
- `K`: option strike.
- `tau`: remaining time to maturity in years.
- `r`: daily risk-free rate input, decimal.
- `q`: continuous dividend-yield proxy.
```

```
- `sigma`: volatility input used for Black-Scholes.
- `bs_price`: theoretical Black-Scholes option value.
- `delta`: Black-Scholes delta.
- `option_type`: `call` or `put`.
- `moneyness`: ATM/OTM label from the episode file.
- `vol_method`: volatility specification. Current values:␣
 ↪{sorted(bs_panel['vol_method'].unique().tolist())}.

## Report resources saved
Figures in `report_figures/`
Tables in `report_tables/`

## Handoff note
Use `delta` as the hedge ratio for the rebalancing simulation.
Filter by `vol_method` to compare hedging performance under different␣
 ↪volatility assumptions.
"""

hedge_readme_text = f"""# Dynamic Delta Hedging Performance Files

Generated by `black_scholes_model.ipynb`, Job C section.

## Main handoff files
- `hedge_performance_series.parquet`
- `hedge_performance_series.csv`
- `report_tables/hedge_summary.csv`
- `report_tables/hedge_rebalance_frequency_summary.csv`

## Simulation convention
The strategy is a short one-option position hedged with `+delta` shares of SPY.␣
 ↪The initial option premium funds the hedge. The cash account accrues at the␣
 ↪risk-free rate between trading dates. On rebalancing dates, the stock␣
 ↪position is adjusted to the current Black-Scholes delta. At expiration, the␣
 ↪stock position is liquidated and the option payoff is settled.

## Rebalancing frequencies
{REBALANCE_SPECS}

## Transaction-cost assumptions, in basis points
{TRANSACTION_COST_BPS_LIST}

## Key series columns
- `target_delta`: Black-Scholes delta used as target stock shares.
- `stock_shares`: post-trade hedge shares.
- `trade_shares`: number of SPY shares bought or sold on that row.
- `cash_account`: cash after accrual, trading, and costs.
- `hedge_asset_value`: cash plus stock value before expiration settlement.
```

```
- `mark_to_market_hedging_error`: hedge asset value minus option mark value.
- `cumulative_transaction_cost`: cumulative estimated trading cost.
- `terminal_hedging_pnl`: final replication error after stock liquidation and␣
 ↪option payoff settlement.

## Interpretation
A terminal hedging P&L close to zero means the discrete hedge replicated the␣
 ↪option payoff well.
Larger positive or negative values indicate hedging error from discrete␣
 ↪rebalancing, volatility input choice, realized price movement, and␣
 ↪transaction costs.
"""

manifest_text = "\n".join([
    "MODEL HANDOFF FILES",
    str(output_parquet),
    str(output_csv),
    str(readme_path),
    "",
    "JOB C HEDGING HANDOFF FILES",
    str(hedge_series_parquet),
    str(hedge_series_csv),
    str(hedge_readme_path),
    str(hedge_summary_csv),
    str(hedge_frequency_csv),
    "",
    "REPORT FIGURES",
    str(FIG_DIR / "fig_delta_paths.png"),
    str(FIG_DIR / "fig_price_paths.png"),
    str(FIG_DIR / "table_volatility_inputs.png"),
    str(FIG_DIR / "table_model_diagnostics.png"),
    str(FIG_DIR / "fig_hedge_error_by_rebalance_frequency.png"),
    str(FIG_DIR / "fig_terminal_hedging_pnl_by_vol_input.png"),
    "",
    "REPORT TABLE CSVS",
    str(TABLE_DIR / "table_volatility_inputs.csv"),
    str(TABLE_DIR / "table_model_diagnostics.csv"),
    str(TABLE_DIR / "bs_delta_summary.csv"),
    str(TABLE_DIR / "episode_summary.csv"),
    str(TABLE_DIR / "hedge_summary.csv"),
    str(TABLE_DIR / "hedge_rebalance_frequency_summary.csv"),
])

readme_path.write_text(readme_text, encoding="utf-8")
hedge_readme_path.write_text(hedge_readme_text, encoding="utf-8")
manifest_path.write_text(manifest_text, encoding="utf-8")
```

```python
print("Saved files:")
for p in [
    output_parquet, output_csv, hedge_series_parquet, hedge_series_csv,
    episode_summary_csv, summary_csv, hedge_summary_csv, hedge_frequency_csv,
    readme_path, hedge_readme_path, manifest_path,
    FIG_DIR / "fig_delta_paths.png",
    FIG_DIR / "fig_price_paths.png",
    FIG_DIR / "table_volatility_inputs.png",
    FIG_DIR / "table_model_diagnostics.png",
    FIG_DIR / "fig_hedge_error_by_rebalance_frequency.png",
    FIG_DIR / "fig_terminal_hedging_pnl_by_vol_input.png",
]:
    print(" -", p)
```

```
Saved files:
 - C:\Users\elain\Downloads\5010-Project\data\output\bs_delta_panel.parquet
 - C:\Users\elain\Downloads\5010-Project\data\output\bs_delta_panel.csv
 - C:\Users\elain\Downloads\5010-
Project\data\output\hedge_performance_series.parquet
 -
C:\Users\elain\Downloads\5010-Project\data\output\hedge_performance_series.csv
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_tables\episode_summary.csv
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_tables\bs_delta_summary.csv
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_tables\hedge_summary.csv
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_tables\hedge_rebalance_frequency_summary.csv
 - C:\Users\elain\Downloads\5010-Project\data\output\bs_delta_panel_README.md
 - C:\Users\elain\Downloads\5010-Project\data\output\hedge_performance_README.md
 -
C:\Users\elain\Downloads\5010-Project\data\output\modeling_outputs_manifest.txt
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_figures\fig_delta_paths.png
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_figures\fig_price_paths.png
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_figures\table_volatility_inputs.png
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_figures\table_model_diagnostics.png
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_figures\fig_hedge_error_by_rebalance_frequency.png
 - C:\Users\elain\Downloads\5010-
Project\data\output\report_figures\fig_terminal_hedging_pnl_by_vol_input.png
```

[23]:
```
check = pd.read_parquet(DATA_PATH / "bs_delta_panel.parquet")
hedge_check = pd.read_parquet(DATA_PATH / "hedge_performance_series.parquet")

print("Reloaded Black-Scholes panel shape:", check.shape)
display(check.head())

print("Reloaded hedging series shape:", hedge_check.shape)
display(hedge_check.head())

print("\nSaved figure files:")
for p in sorted(FIG_DIR.glob("*.png")):
    print(" -", p.name)

print("\nSaved table files:")
for p in sorted(TABLE_DIR.glob("*.csv")):
    print(" -", p.name)
```

```
Reloaded Black-Scholes panel shape: (2847, 19)

       episode_id regime       date           S      K       tau        r       ␣
↪ q   sigma   bs_price     delta  \
0  SPY_calm_C_ATM   calm 2021-03-01  389.579987  390.0  0.241096  0.00028  0.
↪013122  0.2335  16.989992  0.506741
1  SPY_calm_C_ATM   calm 2021-03-02  386.540009  390.0  0.238356  0.00035  0.
↪013122  0.2335  15.399808  0.479406
2  SPY_calm_C_ATM   calm 2021-03-03  381.420013  390.0  0.235616  0.00035  0.
↪013122  0.2335  12.971057  0.432680
3  SPY_calm_C_ATM   calm 2021-03-04  376.700012  390.0  0.232877  0.00028  0.
↪013122  0.2335  10.937231  0.389266
4  SPY_calm_C_ATM   calm 2021-03-05  383.630005  390.0  0.230137  0.00028  0.
↪013122  0.2335  13.758139  0.451908

  option_type cp_flag moneyness        vol_method                            ␣
↪q_source inception_date expiration  \
0        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
↪T_years     2021-03-01 2021-05-28
1        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
↪T_years     2021-03-01 2021-05-28
2        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
↪T_years     2021-03-01 2021-05-28
3        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
↪T_years     2021-03-01 2021-05-28
4        call       C       ATM  iv_inception_vix  div_amount_in_window / S0 /␣
↪T_years     2021-03-01 2021-05-28

           S0
0  389.579987
```

```
1  389.579987
2  389.579987
3  389.579987
4  389.579987

Reloaded hedging series shape: (26055, 32)

       episode_id regime moneyness option_type        vol_method␣
↪rebalance_frequency  rebalance_step_trading_days  \
0  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                              1
1  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                              1
2  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                              1
3  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                              1
4  SPY_calm_C_ATM   calm       ATM        call  iv_inception_vix              ␣
↪daily                              1

   transaction_cost_bps       date    row_type           S      K        r  ␣
↪sigma       tau  option_value  \
0                   0.0 2021-03-01  hedge_mark  389.579987  390.0  0.00028  0.
↪2335  0.241096     16.989992
1                   0.0 2021-03-02  hedge_mark  386.540009  390.0  0.00035  0.
↪2335  0.238356     15.399808
2                   0.0 2021-03-03  hedge_mark  381.420013  390.0  0.00035  0.
↪2335  0.235616     12.971057
3                   0.0 2021-03-04  hedge_mark  376.700012  390.0  0.00028  0.
↪2335  0.232877     10.937231
4                   0.0 2021-03-05  hedge_mark  383.630005  390.0  0.00028  0.
↪2335  0.230137     13.758139

   target_delta  stock_shares  trade_shares  is_rebalance_date  cash_account ␣
↪stock_value  hedge_asset_value  \
0      0.506741      0.506741      0.506741               True   -180.426121  ␣
↪197.416112          16.989992
1      0.479406      0.479406     -0.027335               True   -169.860213  ␣
↪185.309585          15.449372
2      0.432680      0.432680     -0.046726               True   -152.038116  ␣
↪165.032769          12.994653
3      0.389266      0.389266     -0.043414               True   -135.684163  ␣
↪146.636421          10.952258
4      0.451908      0.451908      0.062642               True   -159.715719  ␣
↪173.365481          13.649763
```

```
   mark_to_market_hedging_error  period_transaction_cost ␣
↪cumulative_transaction_cost  cumulative_turnover_dollars  \
0                      0.000000                      0.0                        ␣
↪ 0.0                   197.416112
1                      0.049564                      0.0                        ␣
↪ 0.0                   207.982158
2                      0.023596                      0.0                        ␣
↪ 0.0                   225.804419
3                      0.015026                      0.0                        ␣
↪ 0.0                   242.158517
4                     -0.108376                      0.0                        ␣
↪ 0.0                   266.189968

   n_rebalances_so_far expiration settlement_price_source  terminal_payoff ␣
↪terminal_hedging_pnl
0                    1 2021-05-28                    None              NaN      ␣
↪            NaN
1                    2 2021-05-28                    None              NaN      ␣
↪            NaN
2                    3 2021-05-28                    None              NaN      ␣
↪            NaN
3                    4 2021-05-28                    None              NaN      ␣
↪            NaN
4                    5 2021-05-28                    None              NaN      ␣
↪            NaN


Saved figure files:
 - fig_delta_paths.png
 - fig_hedge_error_by_rebalance_frequency.png
 - fig_price_paths.png
 - fig_terminal_hedging_pnl_by_vol_input.png
 - table_model_diagnostics.png
 - table_volatility_inputs.png

Saved table files:
 - bs_delta_summary.csv
 - episode_summary.csv
 - hedge_rebalance_frequency_summary.csv
 - hedge_summary.csv
 - table_model_diagnostics.csv
 - table_volatility_inputs.csv
```

1.11 11. Suggested team handoff message

I finished the Black–Scholes modeling layer and dynamic delta-hedging simulation. The main modeling handoff file is `data/output/bs_delta_panel.parquet`. The main hedging handoff files are `data/output/hedge_performance_series.parquet` and

`data/output/report_tables/hedge_summary.csv`. The hedging series has one row per episode $\times$ volatility method $\times$ rebalancing rule $\times$ transaction-cost assumption $\times$ date, and it tracks delta targets, stock trades, cash account, transaction costs, mark-to-market hedging error, and terminal hedging P&L. Report figures and tables are saved under `data/output/report_figures/` and `data/output/report_tables/`.